Machine Learning

Lecture 9: Language Model

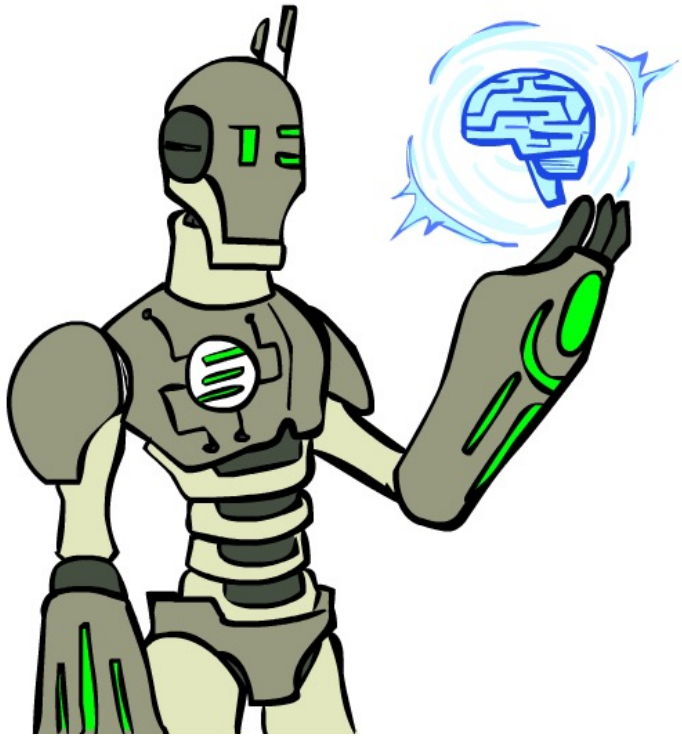
Spring 2025

Instructor: Xiaodong Gu



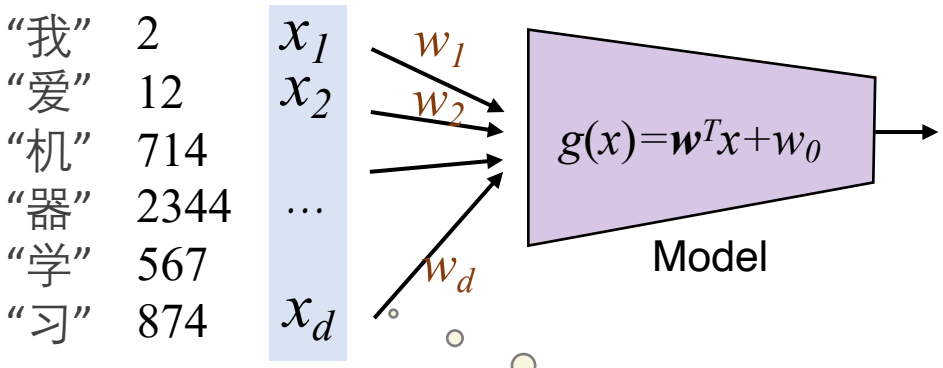
Today

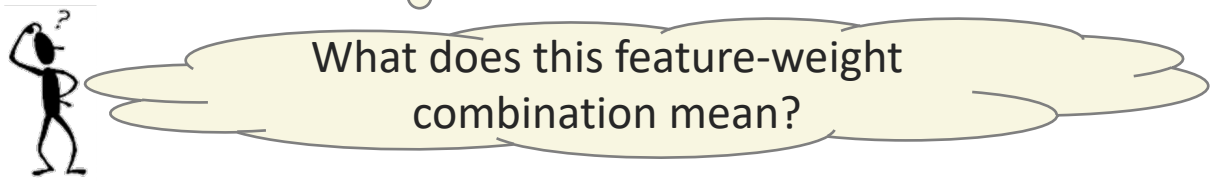
- Word Embedding (word2vec)
- Language Model



Before learning word embedding…

- How to represent words in computers?
  - discrete-symbols (ASCII, id, etc.)
  - e.g., "hotel":11, "conference":81, "motel": 1203, …
  - a localist representation
- How to represent words in machine learning models?





Words as Vectors



- Words can be represented by **one-hot** vectors:

One-Hot Encoding

Means one 1, the rest 0s

hotel = [1 0 0 0 0 0 0 0 0]

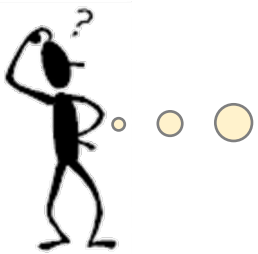
flower = [0 1 0 0 0 0 0 0 0]

tree = [0 0 0 1 0 0 0 0 0]

motel = [0 0 0 0 0 0 0 1 0]

elephant = [0 0 0 0 0 0 0 0 1]

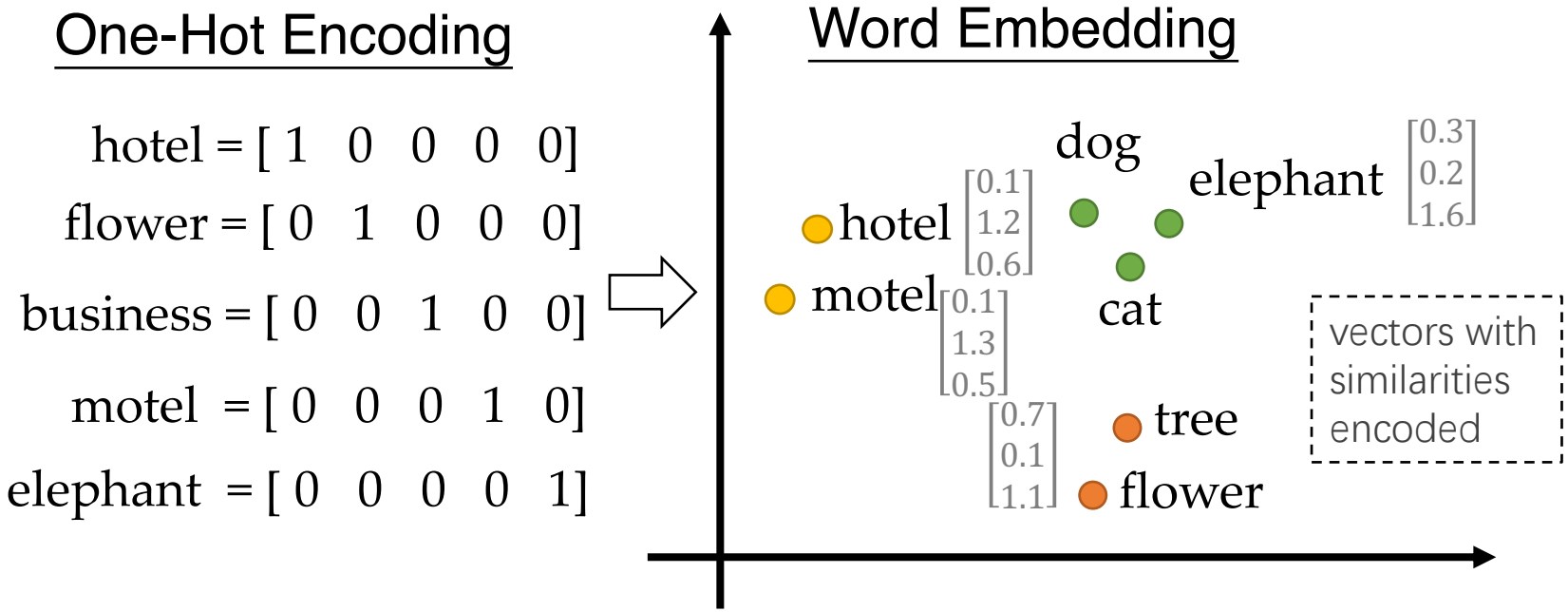
Vector dimensionality = number of words in vocabulary (e.g., **500,000**)



How can we capture relationships (similarity) between words?

Word Embeddings

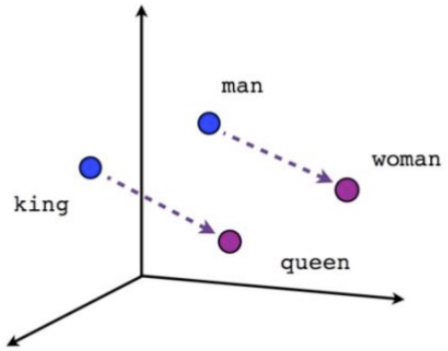
- Represent words as low-dimensional **dense vectors** that can reflect their semantic similarities.



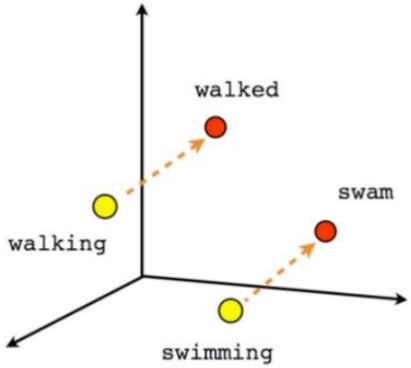
Note: word vectors are sometimes called word embeddings or word representations. They are distributed representations.

Why Word Embeddings?

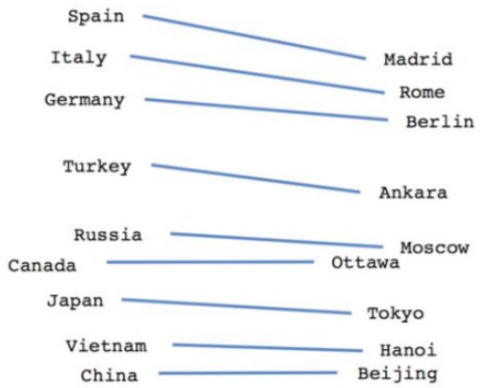
- Can capture the rich relational structure of the lexicon.



Male-Female



Verb tense



Country-Capital

How to obtain word embeddings?

- A word can be understood by its **context**

"A bottle of ***tesgüino*** is on the table"
"Everybody likes ***tesgüino***"
"***Tesgüino*** makes you drunk"
"We make ***tesgüino*** out of corn"

- From context words we can guess **tesgüino** means an alcoholic beverage such as beer.



"You shall know a word by the company it keeps"
(J. R. Firth 1957)

- Intuition for an algorithm:



Two words are similar if they have similar word contexts

Word2Vec: Overview



- **Word2vec** (Mikolov et al. 2013) is a framework for **learning** word vectors.



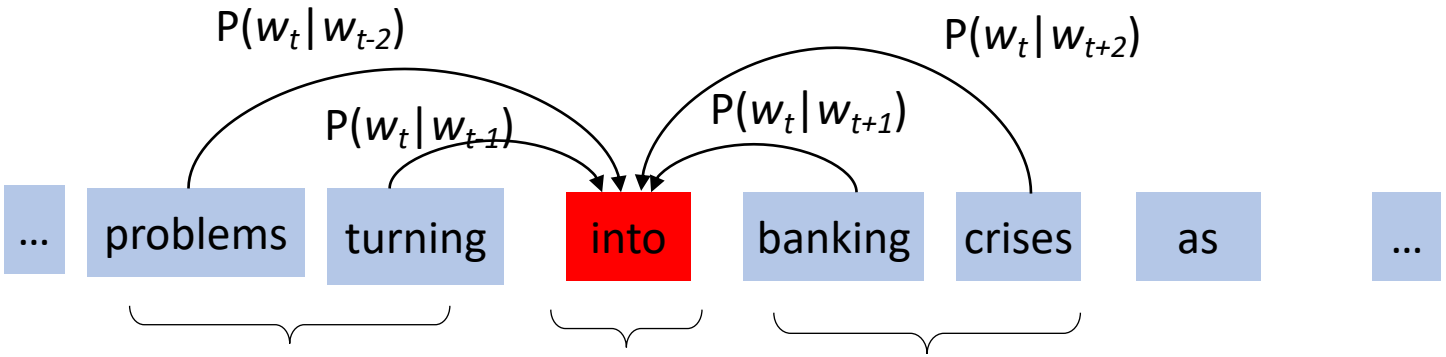
Idea:

- we have a large corpus of text.
- every word in a vocabulary is assigned with a **learnable vector**;
- go through each position t in the text, which has a center word w and context words c;
- calculate the probability of w given c using their vector similarities;
- keep updating the word vectors to maximize this probability.

Mikolov T, Sutskever I, Chen K, et al. Distributed representations of words and phrases and their compositionality. NIPS 2013.

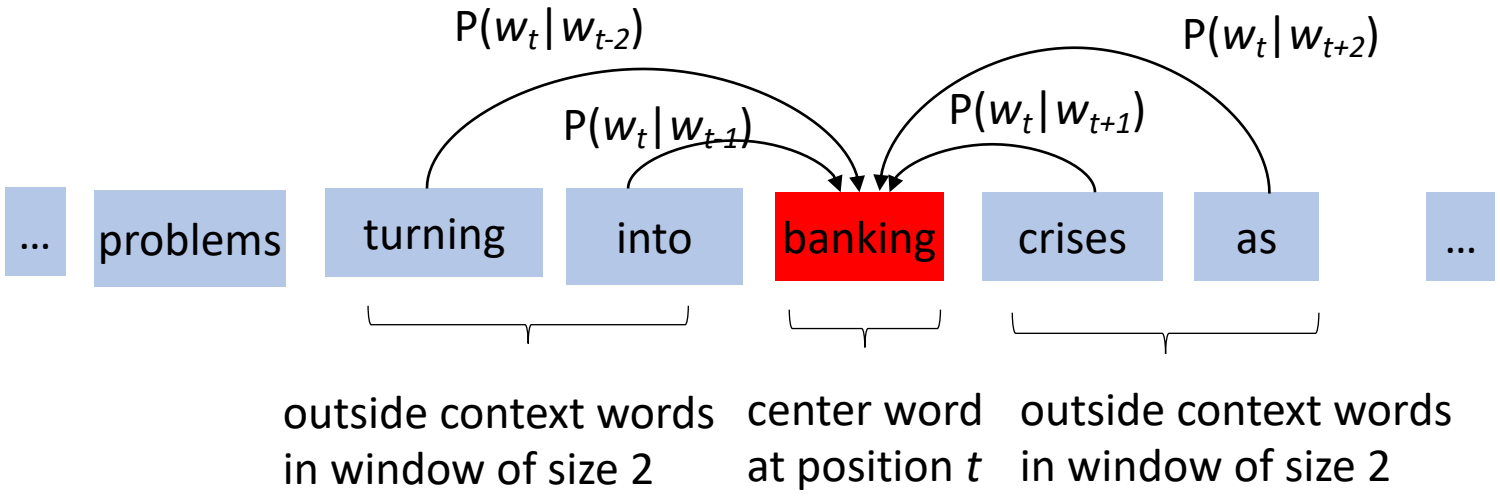
Word2Vec: Overview

Example: window and process for computing $P(w_t | w_{t+j})$



Word2Vec: Overview

Example: window and process for computing $P(w_t | w_{t+j})$



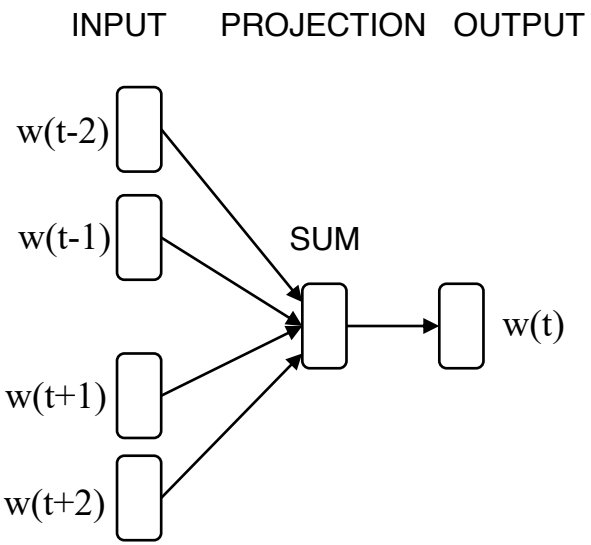
Mikolov's CBOW

- **CBOW**: the distributed representations of context (or surrounding words) are combined to **predict the word in the middle.**

$$P(w_t \mid w_{t-k}, \ldots w_{t-1}\ w_{t+1} \ldots w_{t+k}) = \text{softmax}\ (\mathbf{NN}\ (V(w_{t-k}) + \ldots + V(w_{t+k}))\ )$$

This can be represented as a **neural network**:

- An input layer which converts each word (one-hot) into a dense vector.
- A projection layer which aggregates the vectors of input words.
- An output layer which predicts the target word w_t given the aggregated context vector.

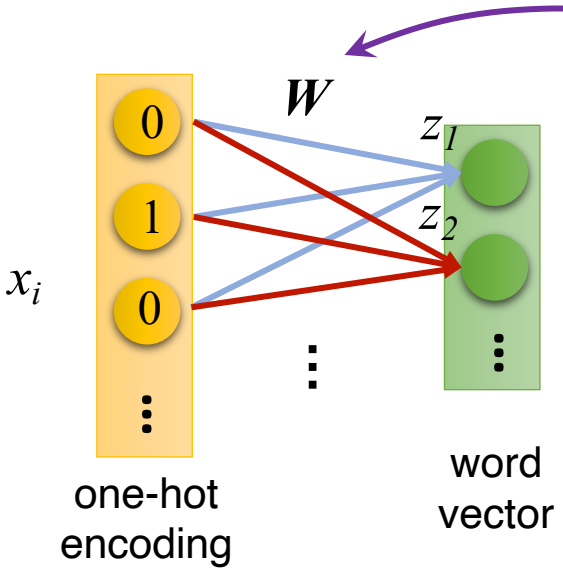


CBOW = Continuous-Bag-of-Words

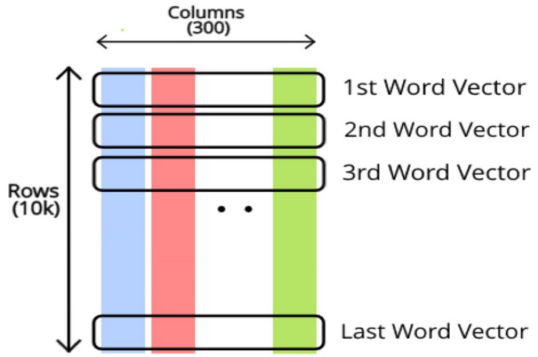
The order of context words has no impact to the projection.

Model Architecture

- **Input Layer:** represent any word into a vector. $z_i = \boldsymbol{W}x_i = \boldsymbol{W}_i$



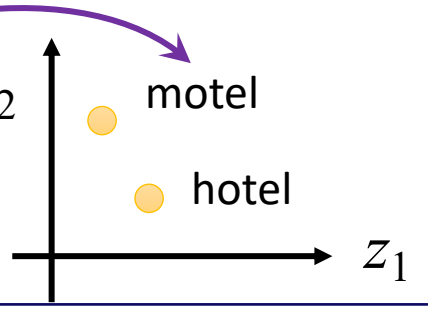
The weight matrix $\boldsymbol{W} \in R^{|V| \times d}$ is a **lookup table** with each row $\boldsymbol{W}_i$ being the vector for word w_i.



Example

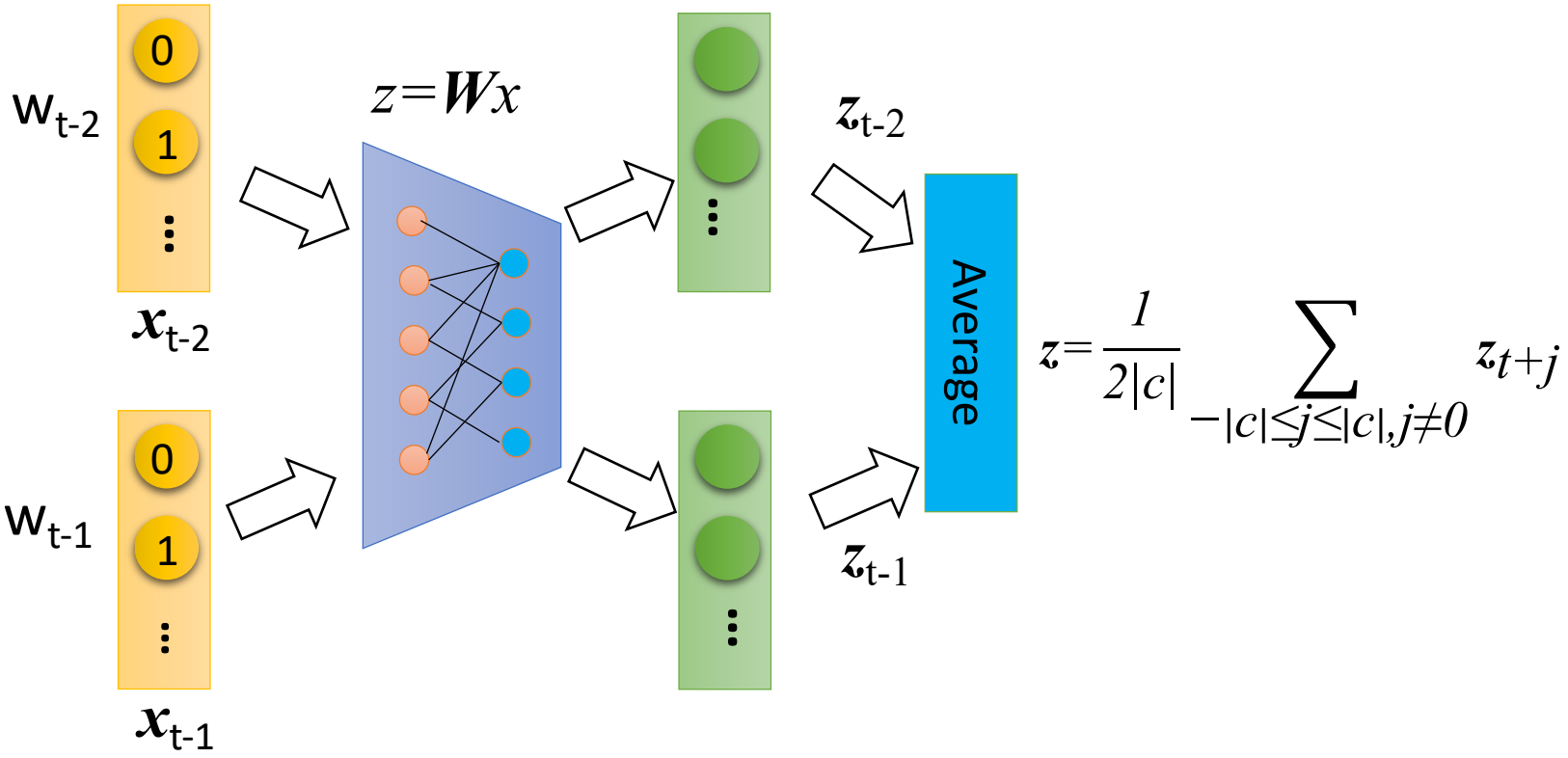
$$[0 \quad 0 \quad 0 \quad \boxed{1} \quad 0] \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ \boxed{10} & \boxed{12} & \boxed{19} \\ 11 & 18 & 25 \end{bmatrix} = [10 \quad 12 \quad 19]$$

"motel"



Architecture

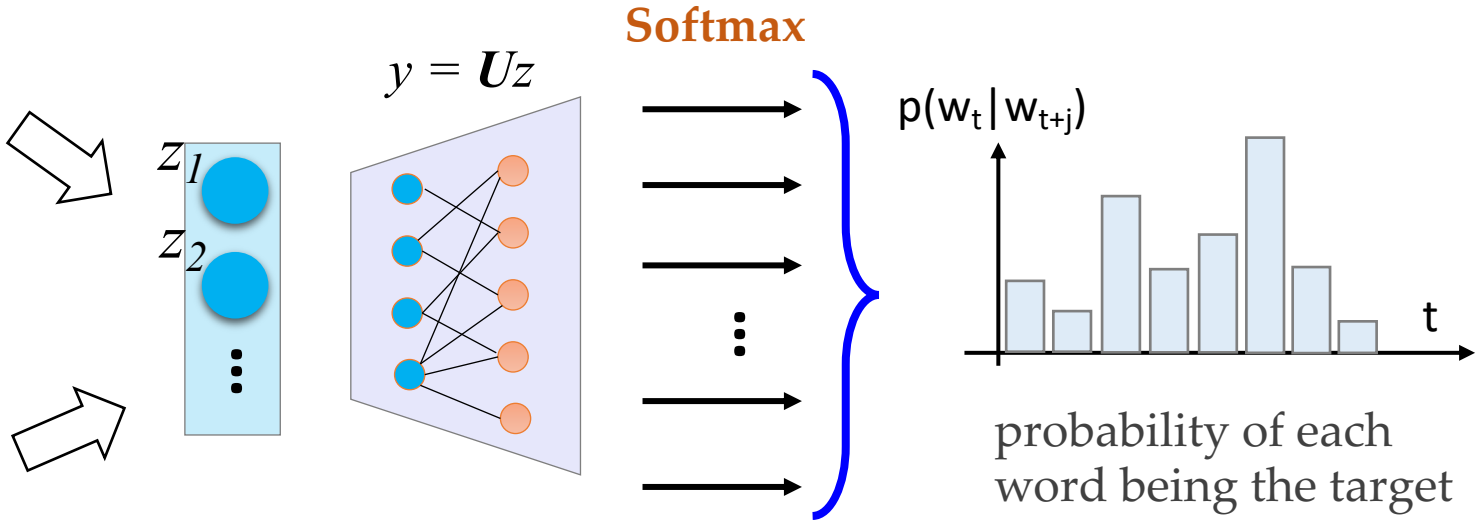
- **Projection Layer**: aggregate context vectors into one vector.



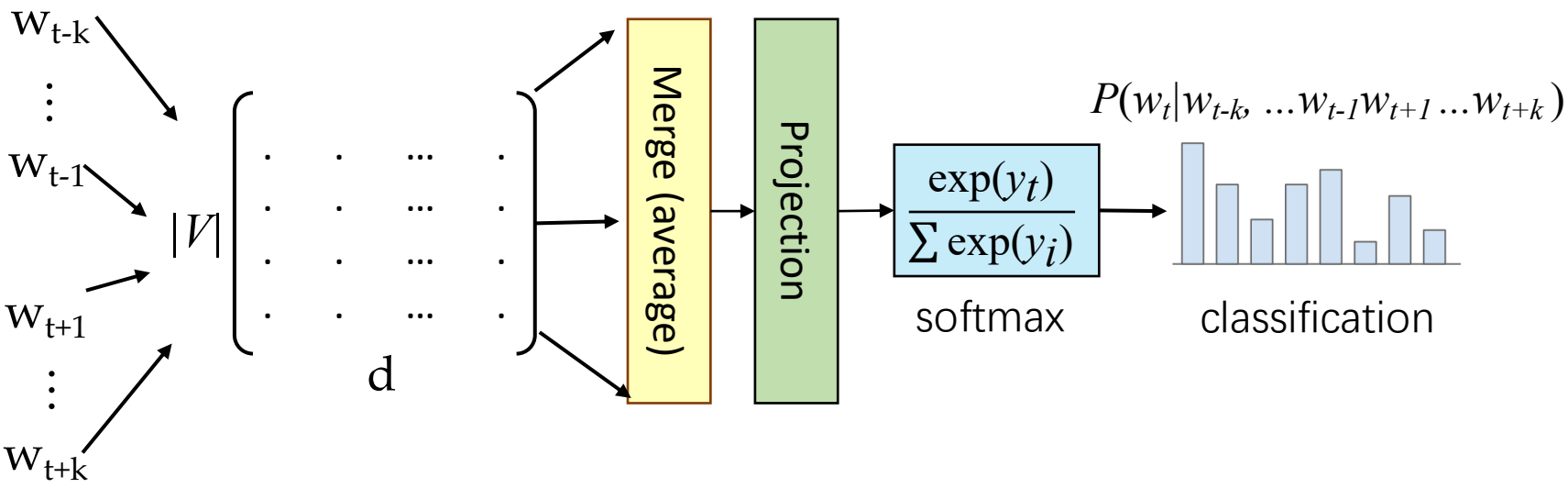
Architecture

- **Output Layer**: predicts the probability of the target word.

$$P\left(w_t \middle| w_{t-|c|}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+|c|}\right) = \frac{\exp\left(y_t\right)}{\sum_{i=1}^{|V|} \exp\left(y_i\right)}$$



The Big Picture



Training



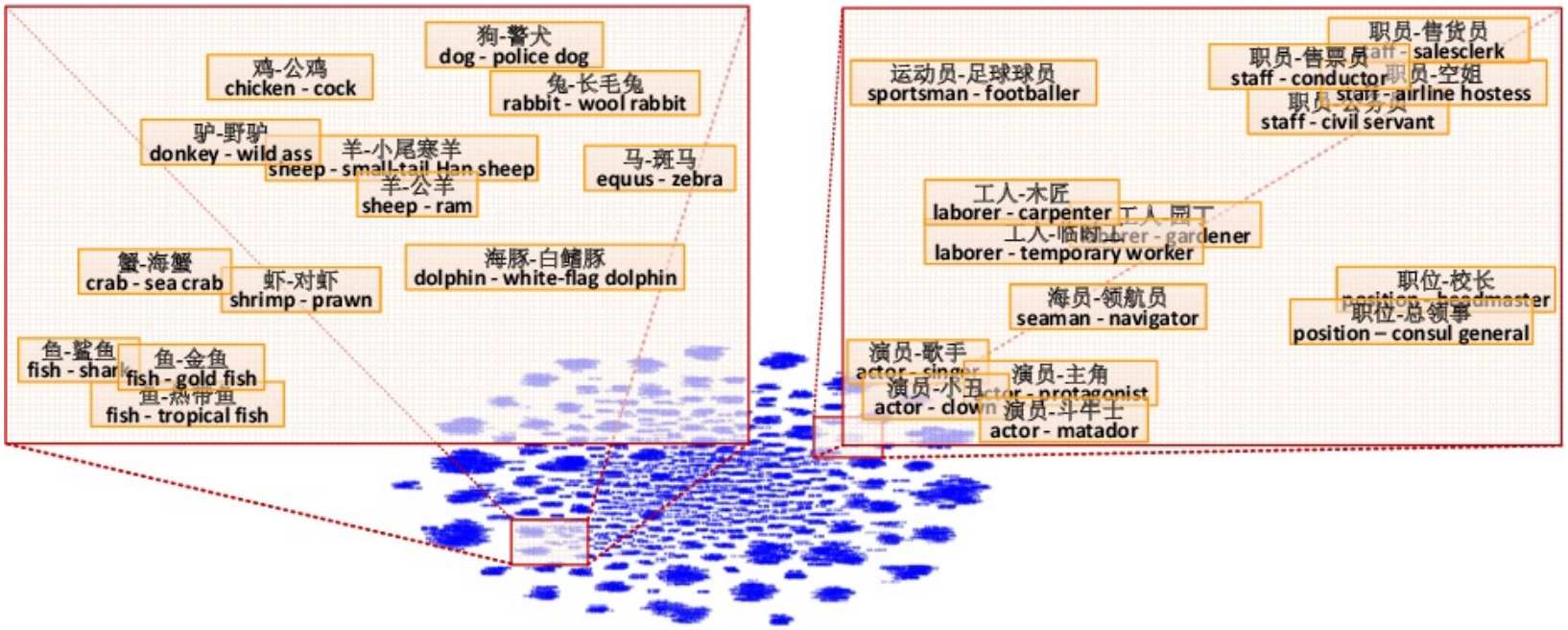
- Given $D = \{w_1, w_2, \ldots, w_N\}$, minimize the **negative log likelihood** (NLL) loss function:

$$L(\boldsymbol{W}, \boldsymbol{U} \mid D) = -\frac{1}{N}\sum_{t=1}^{N} \log p(w_t | w_{t-k}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+k})$$

using gradient descend.

Example

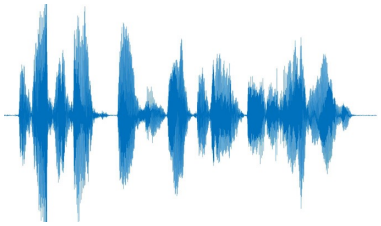
- Word2vec maximizes an objective function by putting similar words nearby in space.



Fu, Ruiji, et al. "Learning semantic hierarchies via word embeddings."*ACL* 2014.

From Words to Texts

- Audio:



- Word:

自然，语言

- Sentence:

我爱自然语言处理

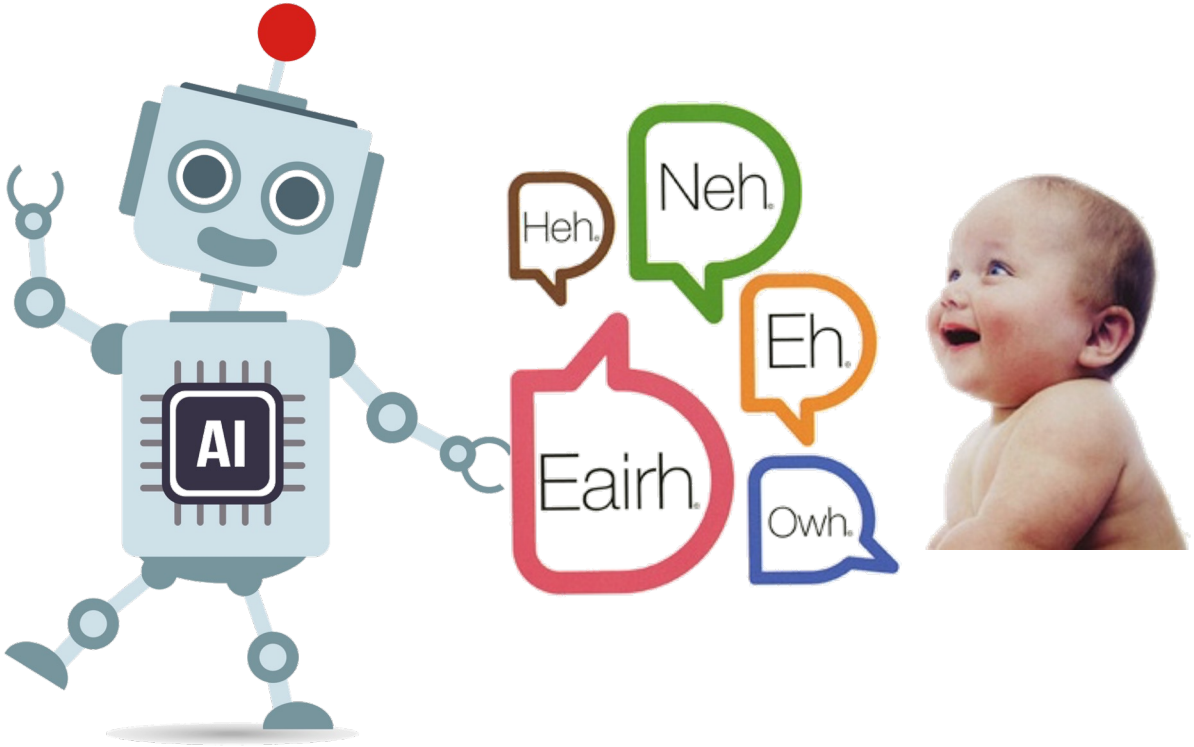
- Program:

```
for (int i = 0; i < 10 ; i ++ ) {
```

…

How can computers speak natural languages?

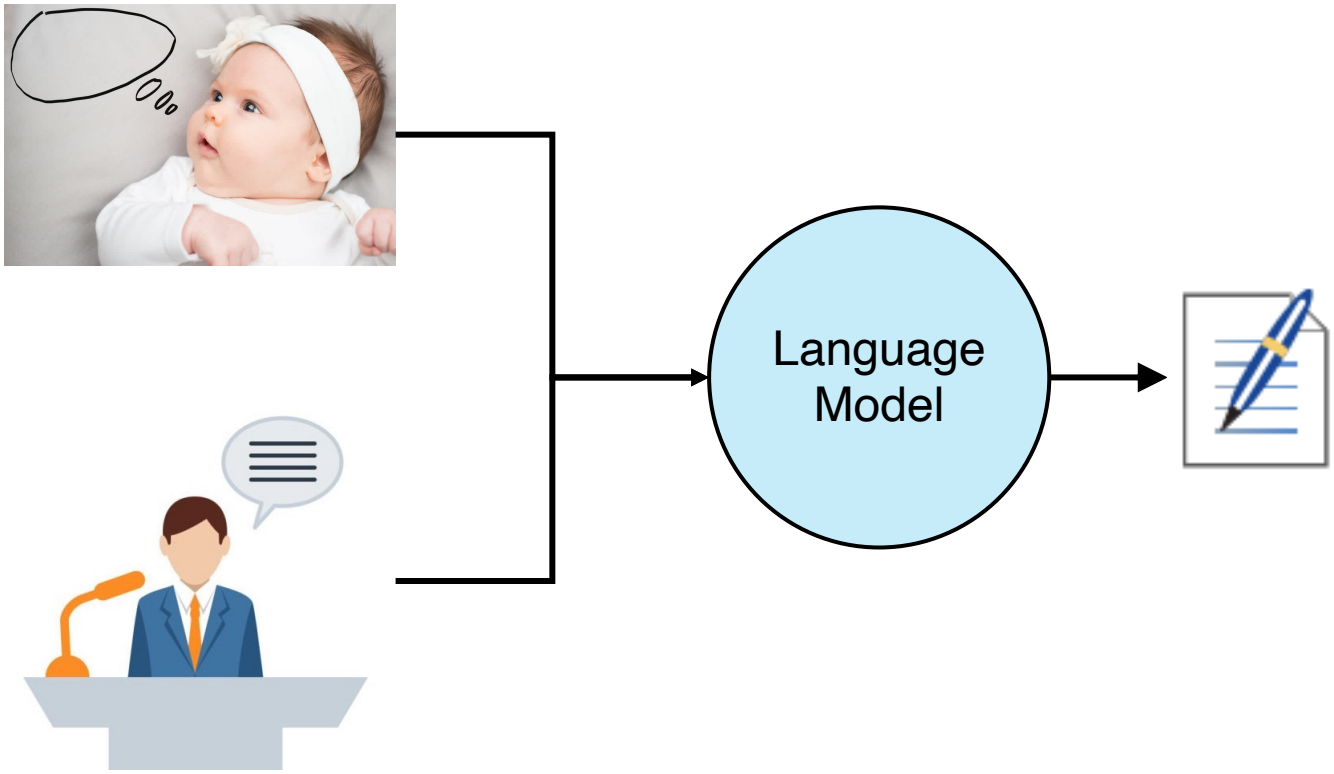
- Teach computers to talk like a baby?



But wait, how to measure the linguistic ability of computers?

Language Model

- How proficiency do you acquire a language?



Language Model



- Which is more standard English?

  "Long time no see" vs "It has been a long time since I saw you"

- How about

  Give you some color see see

  Give me some money buy buy

  …

Language Model



- A probabilistic model of **how likely** a given string appear in a given "**language**".
- For any sequence $x = (w_1, w_2, \ldots, w_N)$, a **language model** can be defined as:

$$p(x) = p(w_1, w_2, \ldots, w_N)$$

Example:

P_1=P("他明天去上课")
P_2=P("明天他将要会去上课")
P_3=P("明儿个他上课")
P_4=P("侬明朝得返课啦，唔好迟到啊嘅 ")

Chinese:
$P_1 > P_2 > P_3 > P_4$

- Applications: 他明天将去上____？__

message suggestion; document generation; spelling correction; machine translation; speech recognition;…

Language Model

- What is the probability of $P(w_1, \ldots, w_N)$?

p(我爱机器学习) = ?

- Chain Rule:

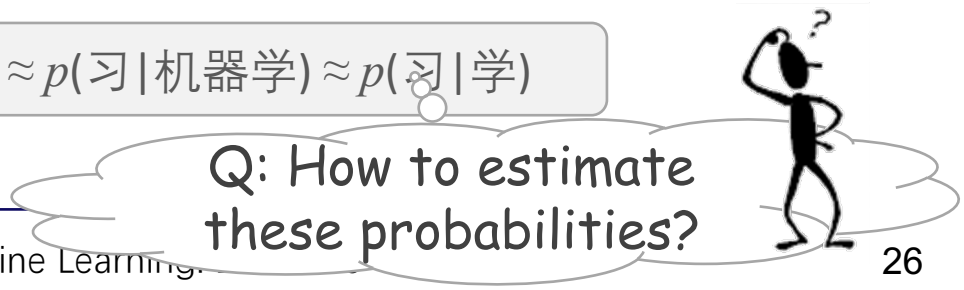
$$p(w_1,\ldots,w_N) = p(w_1)p(w_2|w_1)\ldots p(w_N|w_1,\ldots,w_{N-1})$$

p(我爱机器学习) = p(我)p(爱|我)p(机|我爱)p(器|我爱机)p(学|我爱机器)p(习|我爱机器学)

- Markov Assumption: (only consider the last n -1 words)

$$p(w_i|w_1,\ldots,w_{i-1}) = p(w_i|w_{i-n+1},\ldots,w_{i-1})$$

p(习|我爱机器学) ≈ p(习|机器学) ≈ p(习|学)



Language Model = Predicting the Next word

$$p(w_1, \ldots, w_N) \approx p(w_1)p(w_2|w_1)\ldots p(w_N|w_1, \ldots, w_{t-1})$$

How to estimate $p(w_t|w_1,\ldots,w_{t-1})$?

- **The core objective of an LM:**
  - Given a sequence of t-1 words $w_1, \ldots, w_{t-1}$
  - Predict the next token $p(w_t|w_1, \ldots, w_{t-1})$

“天 将 降 大 任 于 是”

given these words

predict the next word

A Sequence Modeling Problem



- Idea #1: Counting?

“天 将 降 大 任 于 是”

given these words

predict the next word

$$p(\text{是} \mid \text{天将降大任于}) = \frac{\textbf{count}\,(\text{天将降大任于是})}{\textbf{count}\,(\text{天将降大任于})}$$

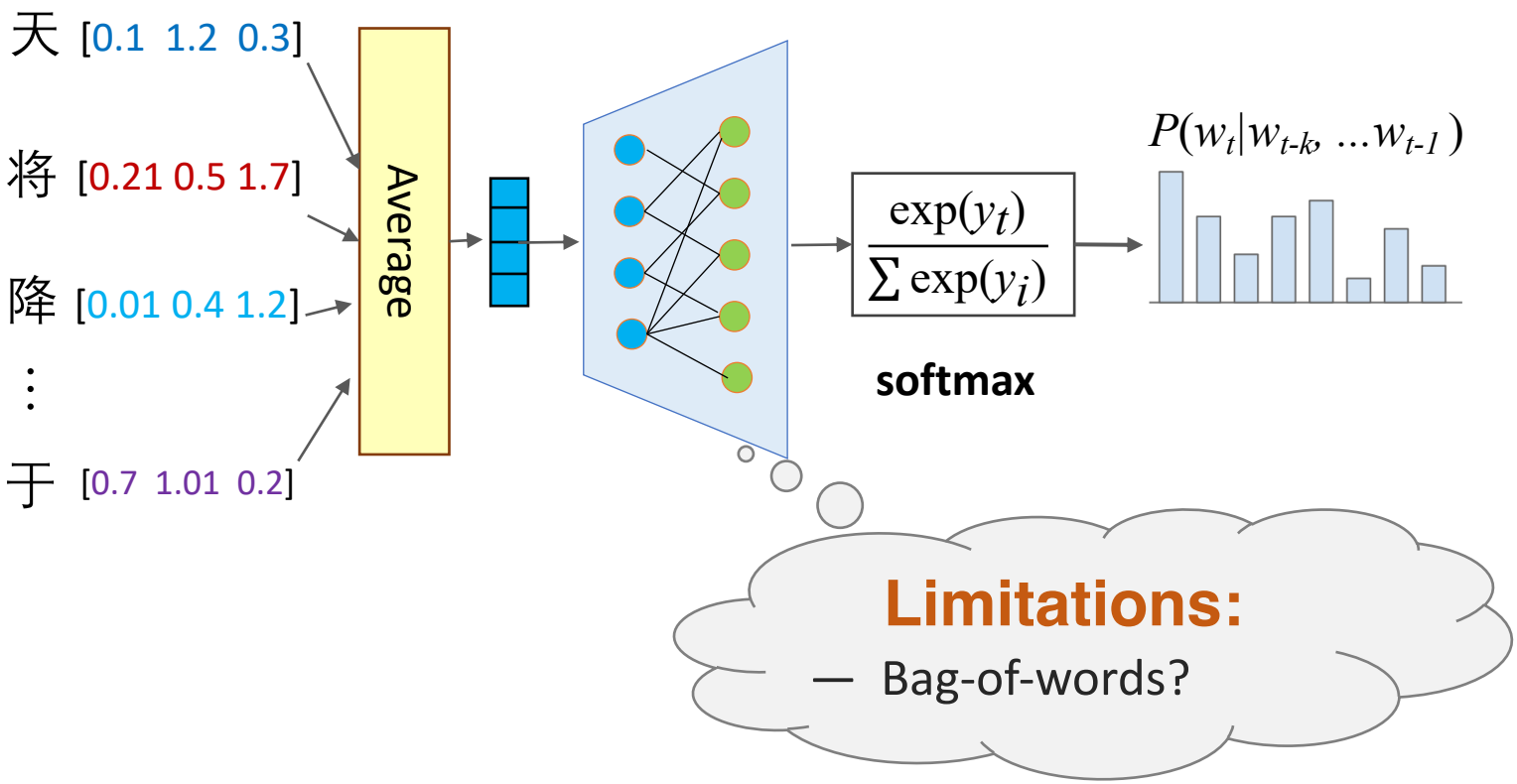
Problems:

- Discrete symbols,
- No word semantics,
- Insensitive to similar words and sentences.

How about “天欲把重担给__”?

Neural Networks as Language Models

- Idea #2: Averaging word vectors

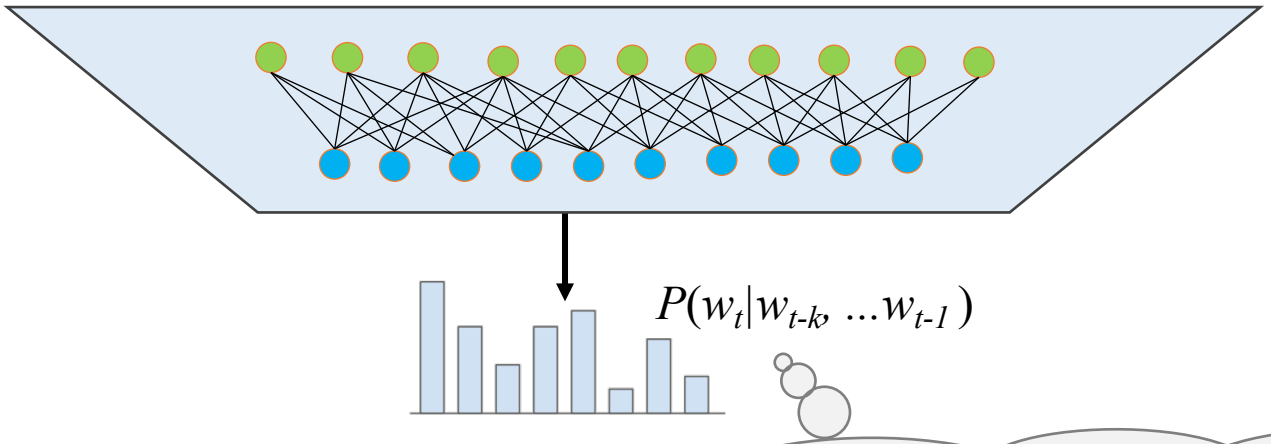


Neural Networks as Language Models

- Idea #3: Concatenate word vectors

天 将 降 大 任

[0.1 1.2 0.3 0.2 0.5 1.7 0.01 0.4 1.12 0.7 1.0 0 0.5 1.1 0.5 …]



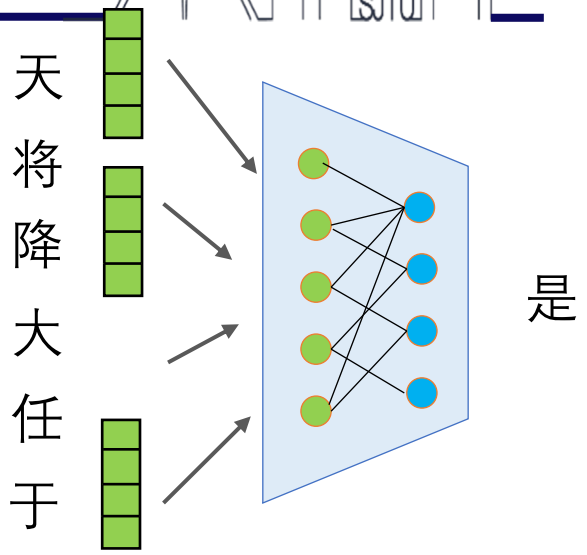
Limitations:

— Cannot handle variant input length?

— Cannot share parameters

Sequence Modeling: Design Criteria

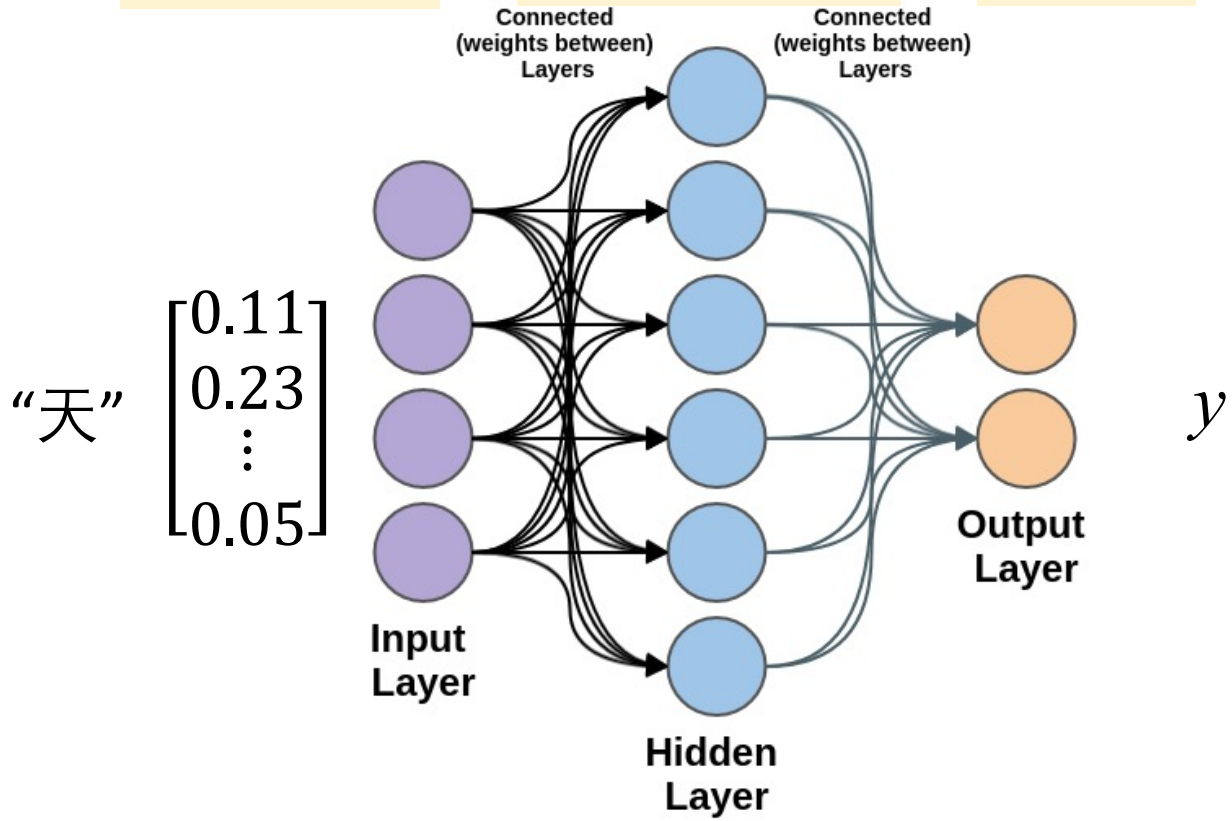
- To model sequences, we need to:
  1. Handle variable-length sequences
  2. Track long-term dependencies
  3. Maintain information about order
  4. Share parameters across the sequence



- **Recurrent Neural Networks** (RNNs) as an ideal approach for sequence modeling.

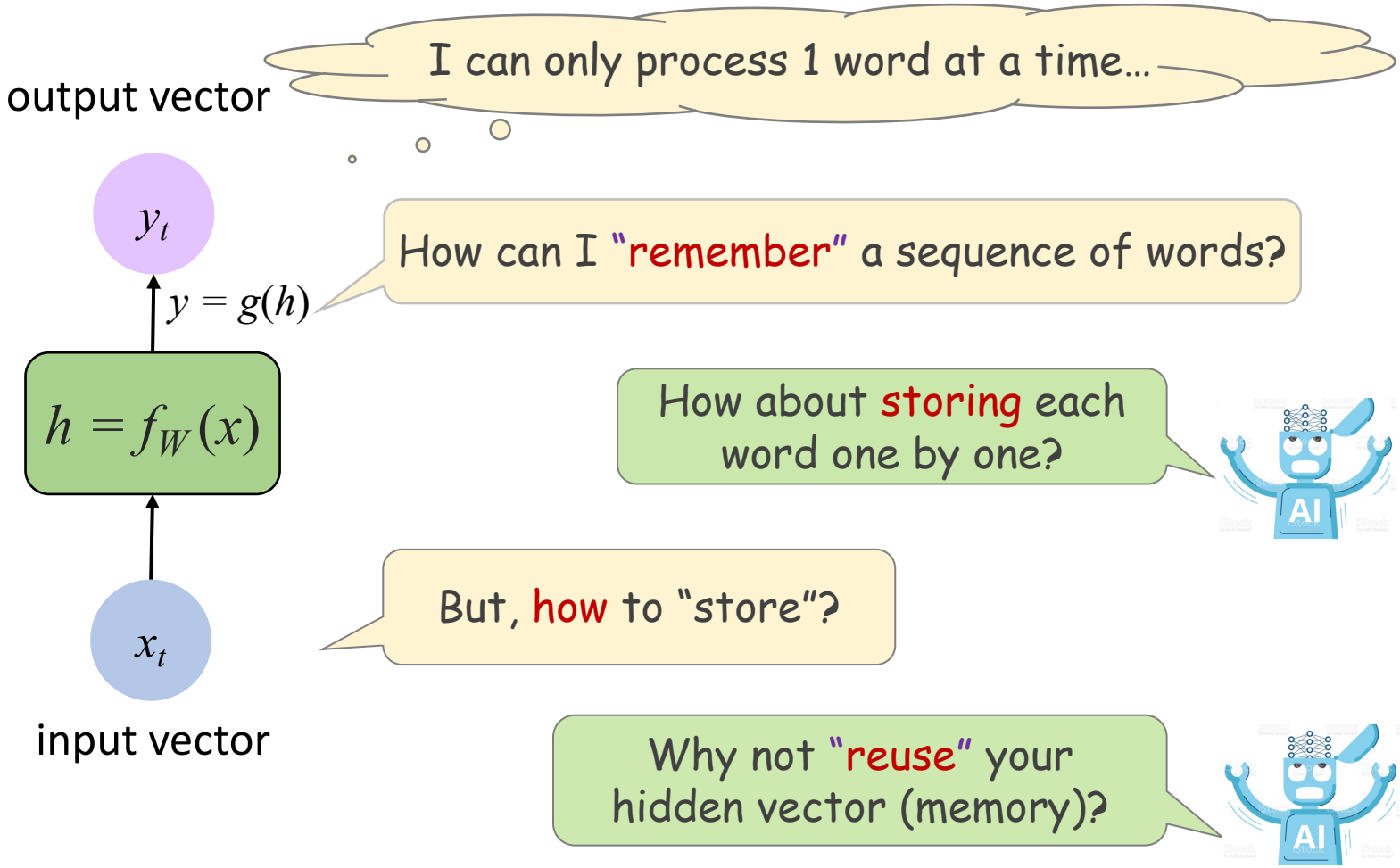
Standard Neural Networks Revisited

Information | Hidden State | Output



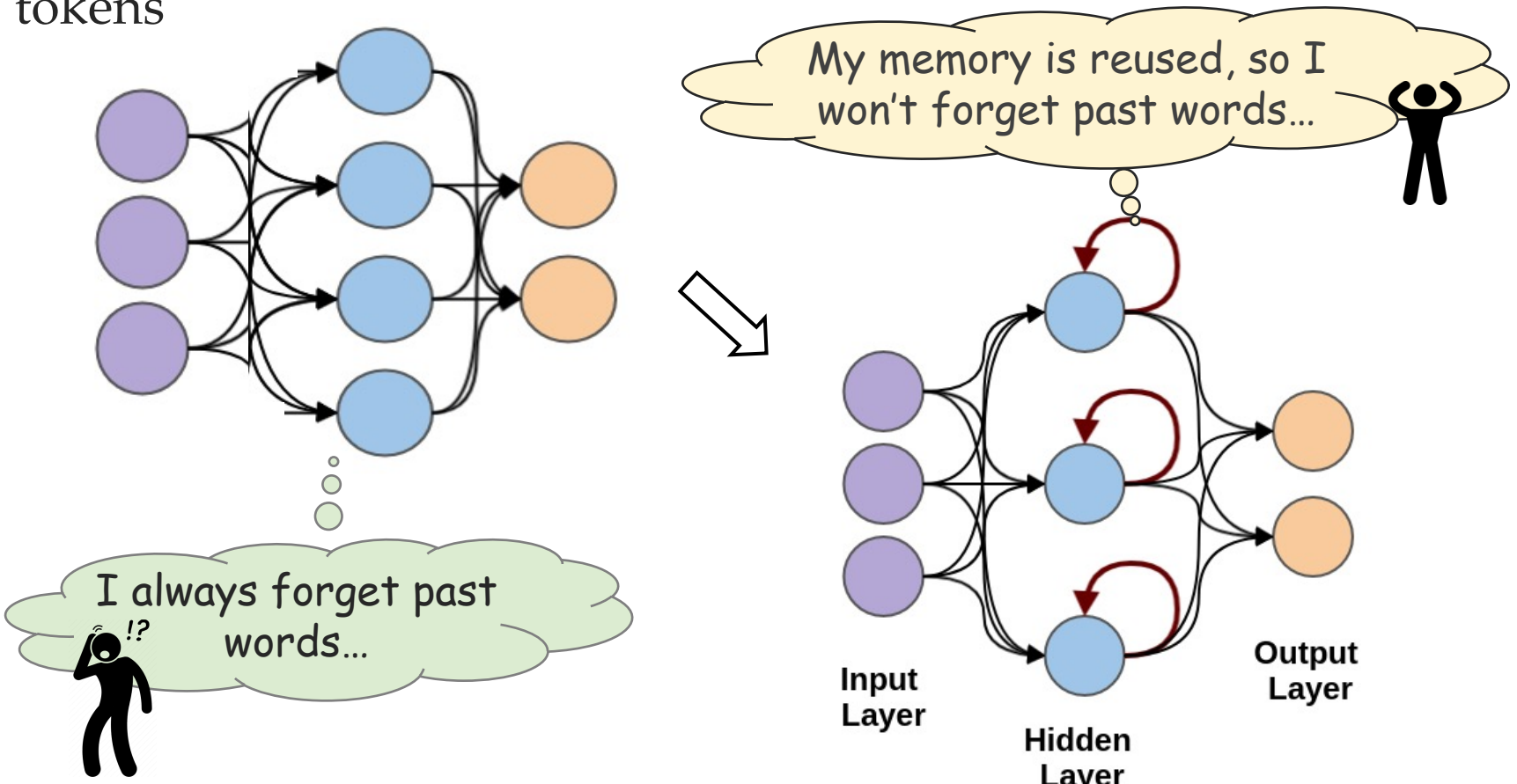
$x \in R^d$ $\quad h = \tanh(W^T x)$ $\quad y = Vh$

Standard "Vanilla" Neural Network

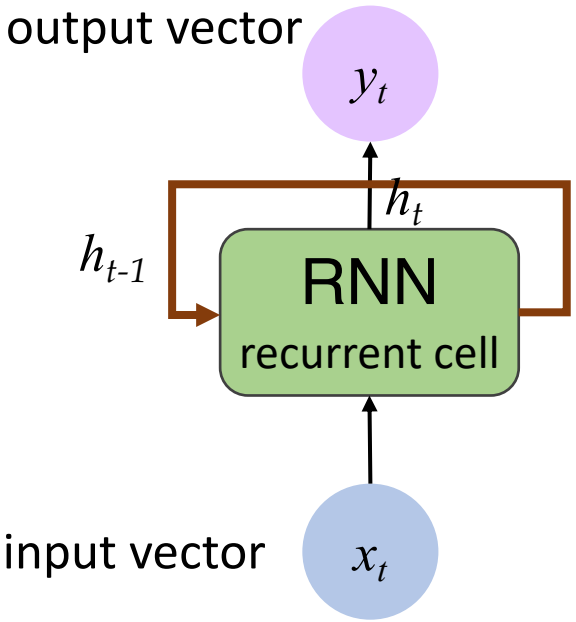


Recurrent Neural Networks

Idea: reuse the hidden layer to memorize the past input tokens



Recurrent Neural Network (RNN)



Apply a recurrent relation at every time step to process a sequence:

$$h_t = f_W(h_{t-1}, x_t)$$

cell state

function parameterized by W

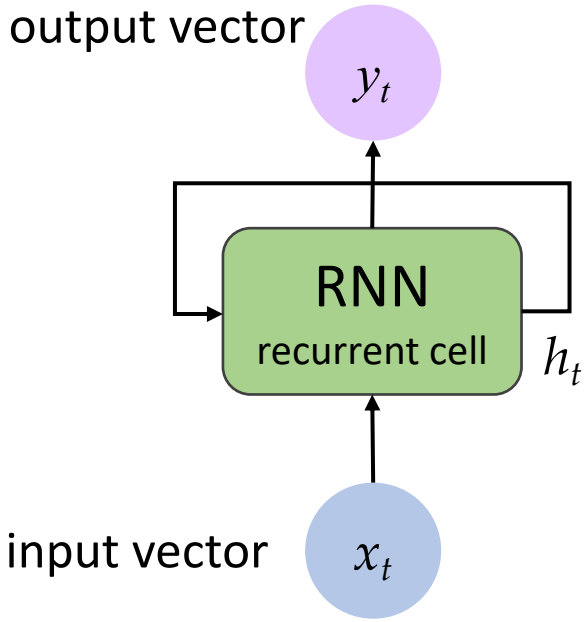
old state

Input vector at time step t

Note: the same function and set of parameters are used at every time step

RNNs have a **state**, h_t, that is updated **at each time step** as a sequence is processed.

RNN State Update and Output



Output Vector

$$y_t = \boldsymbol{W}_{hy}^{T} h_t$$

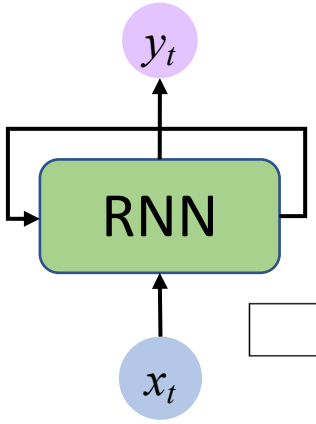
Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{\text{hh}}^{\text{T}} h_{t-1} + \boldsymbol{W}_{\text{xh}}^{\text{T}} x_t)$$

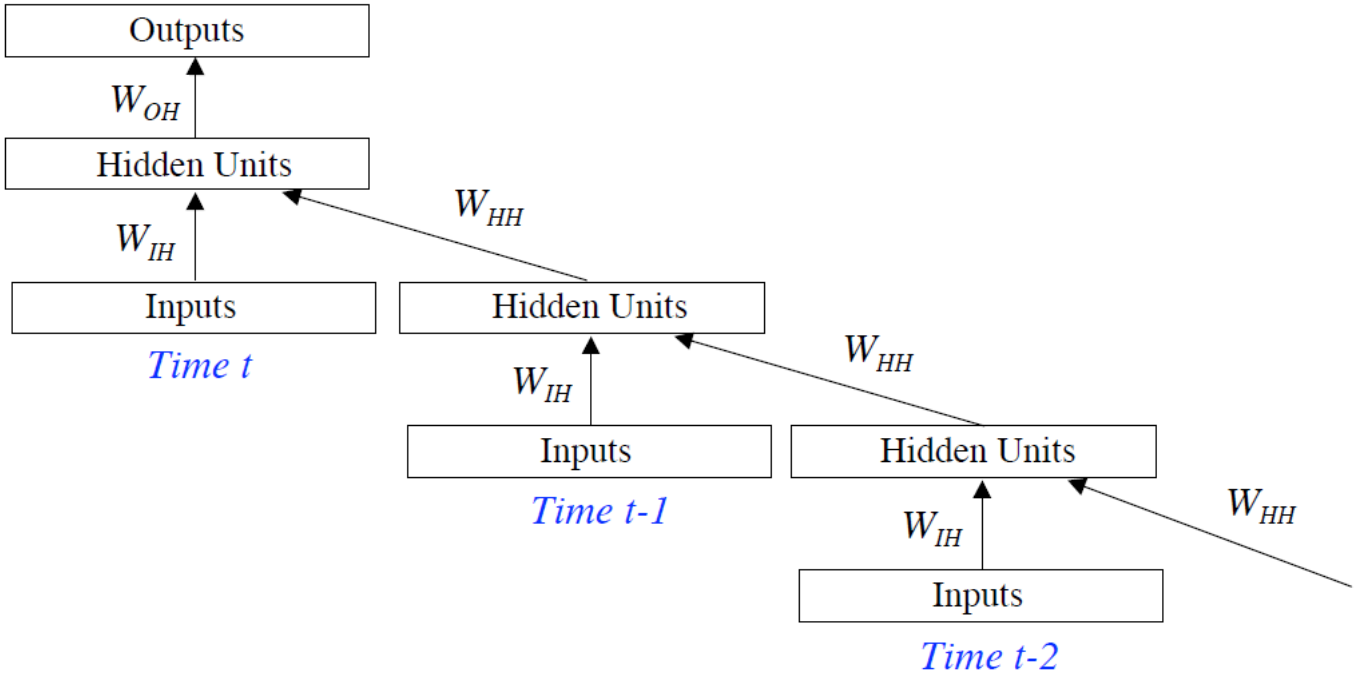
Input Vector

$$x_t$$

RNNs: Computational Graph Across Time

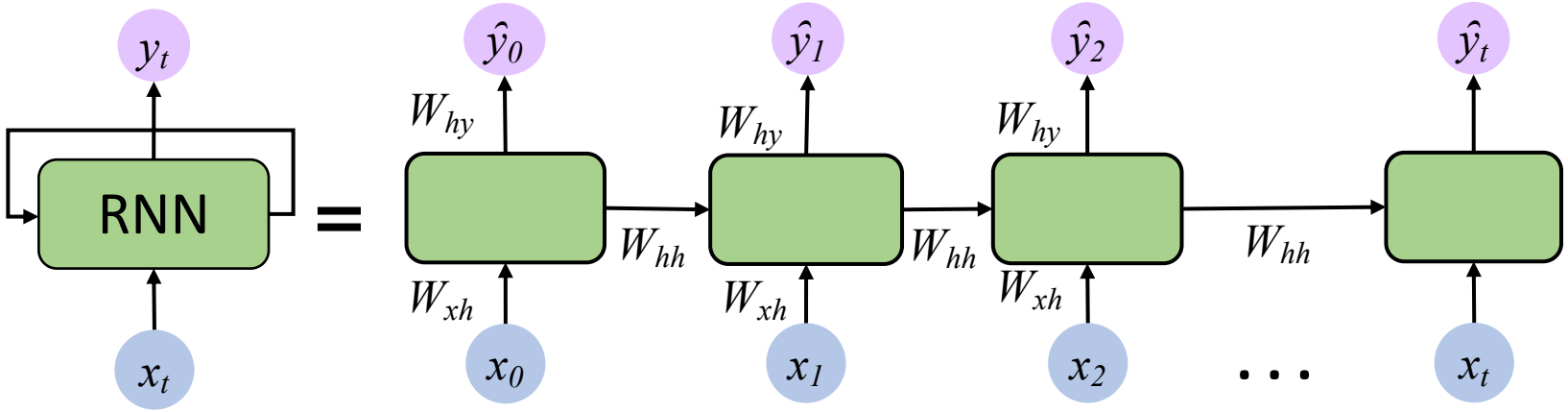


= Represent as computational graph unrolled across time



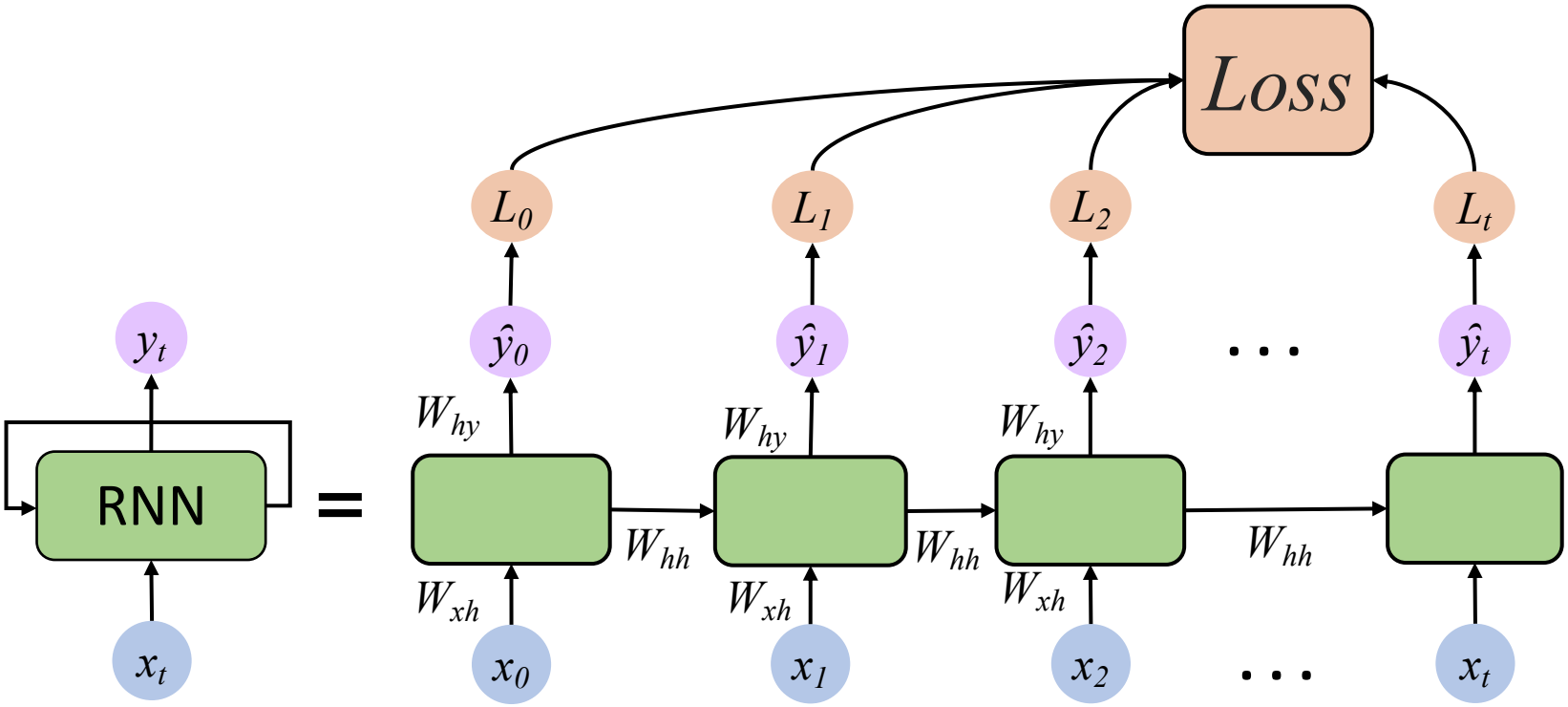
RNNs: Computational Graph Across Time

- Re-use the same weight matrices at every time step



RNNs: Computational Graph Across Time

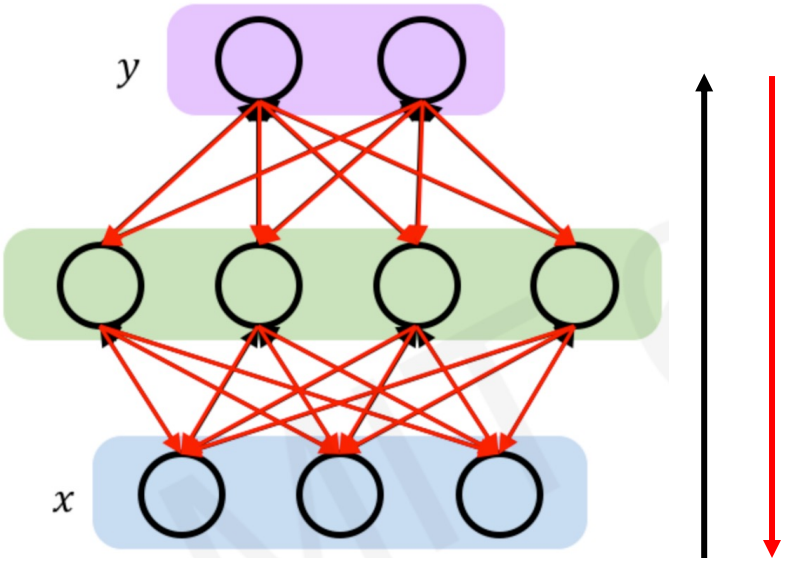
→ forward pass





Backpropagation Through Time (BPTT)

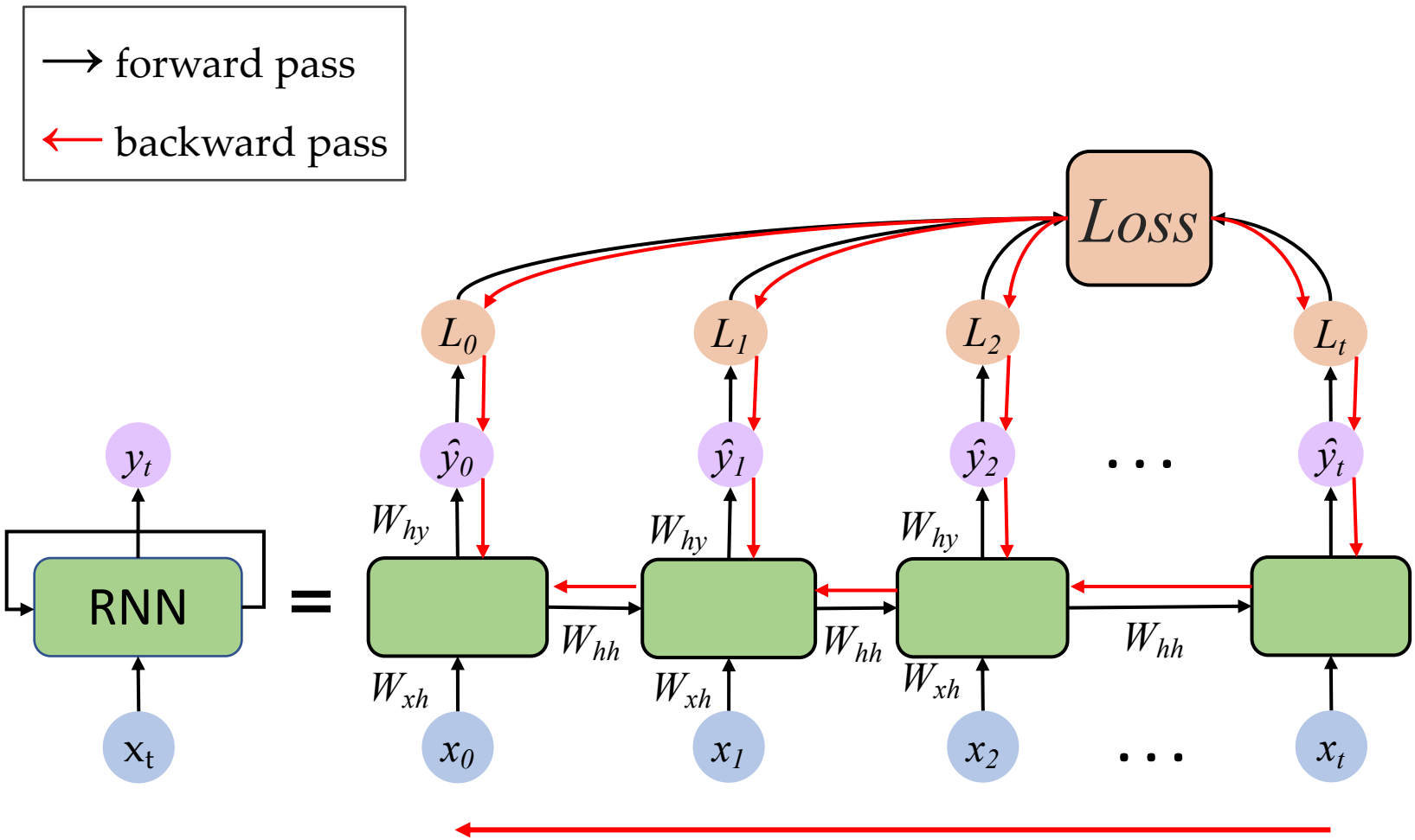
Recall: Backpropagation in Feed Forward NNs



Backpropagation algorithm:

1. Take the derivative (gradient) of the loss with respect to each parameter
2. Shift parameters in order to minimize loss

RNNs: Backpropagation Through Time



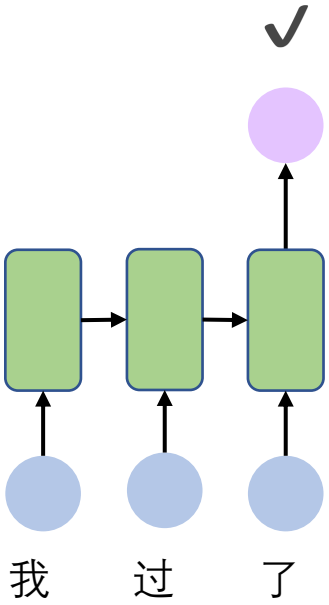


RNN Applications

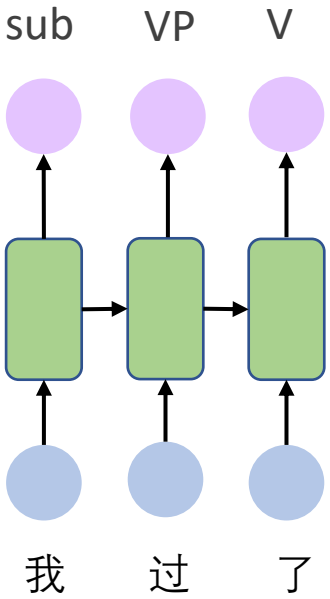
RNNs for Sequence Modeling



One to One
"Vanilla" neural network



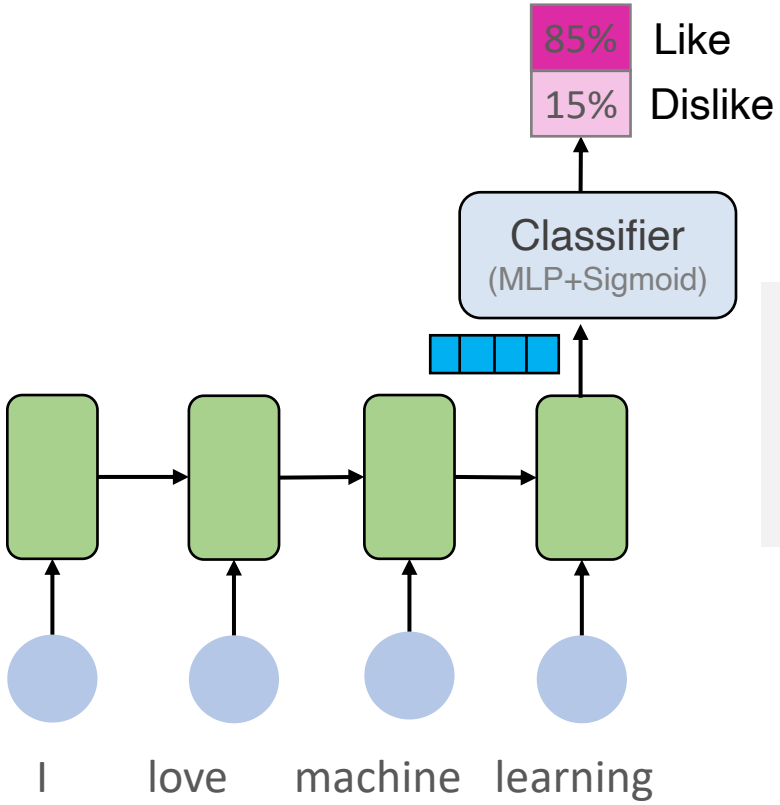
Many to One
e.g., sentiment classification



Many to Many
e.g., information extraction

…and many other architectures and applications

RNN for Sequence Classification



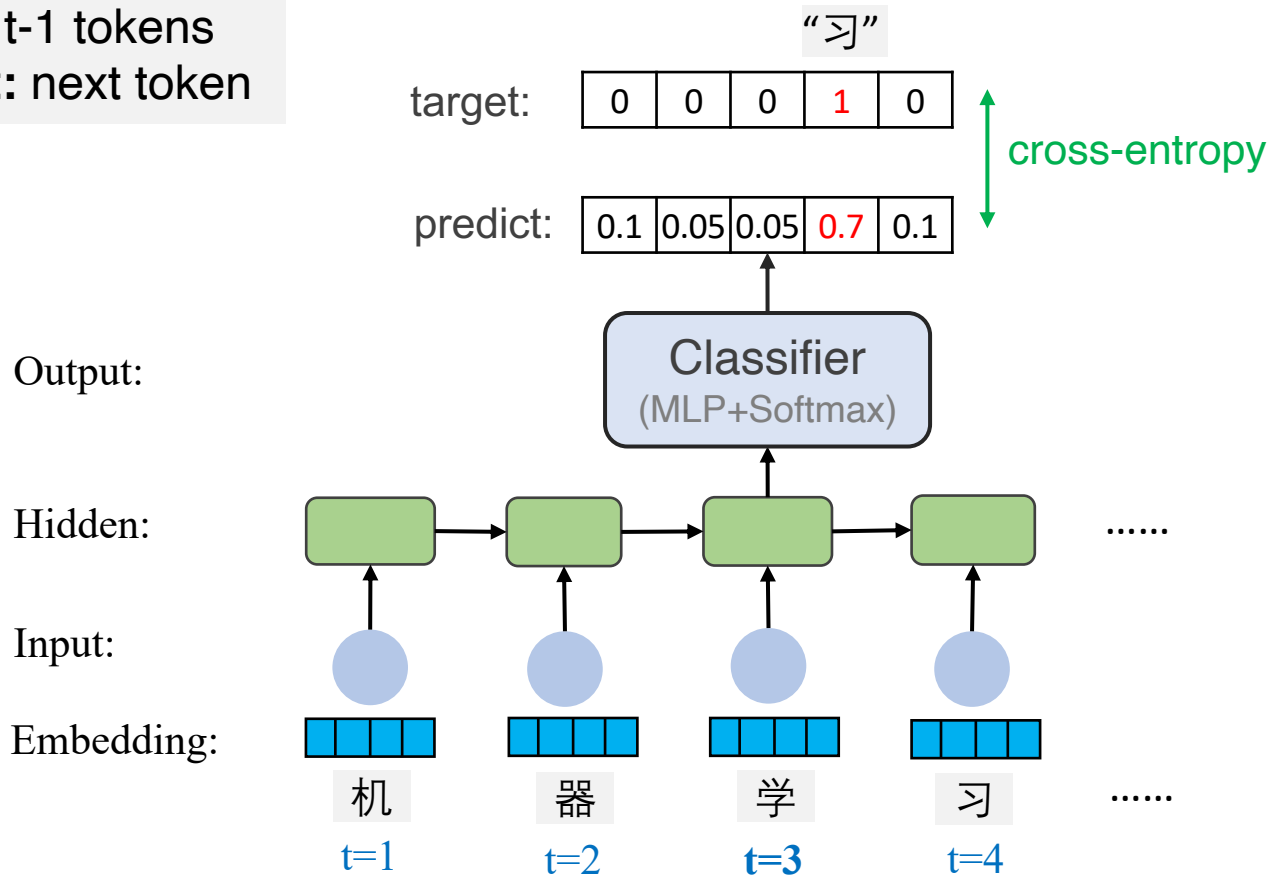
Input: a sequence of words.

Output: probability of each class that the sequence belongs to

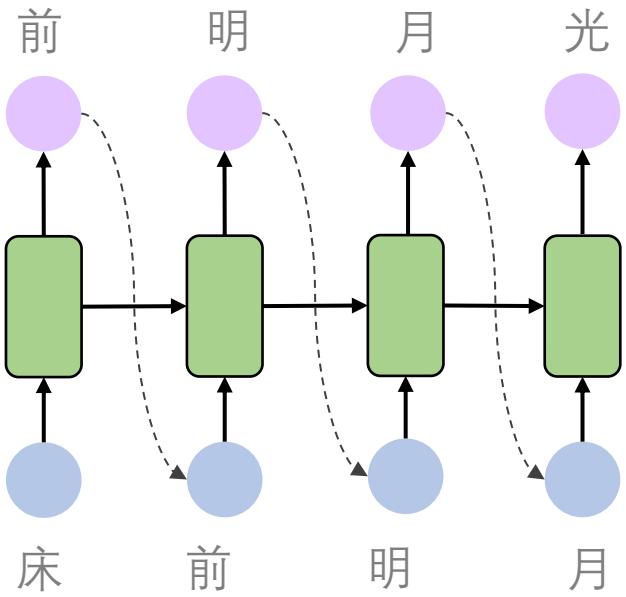
RNN for Sequence Generation

- RNN can also be used to generate a sequence.

Input: t-1 tokens
Output: next token



Example Task: Poetry Generation



Input: n-1 poetry characters

Output: next character in a poetry

TIME for Coding

Tutorial: poetry generation using Pytorch

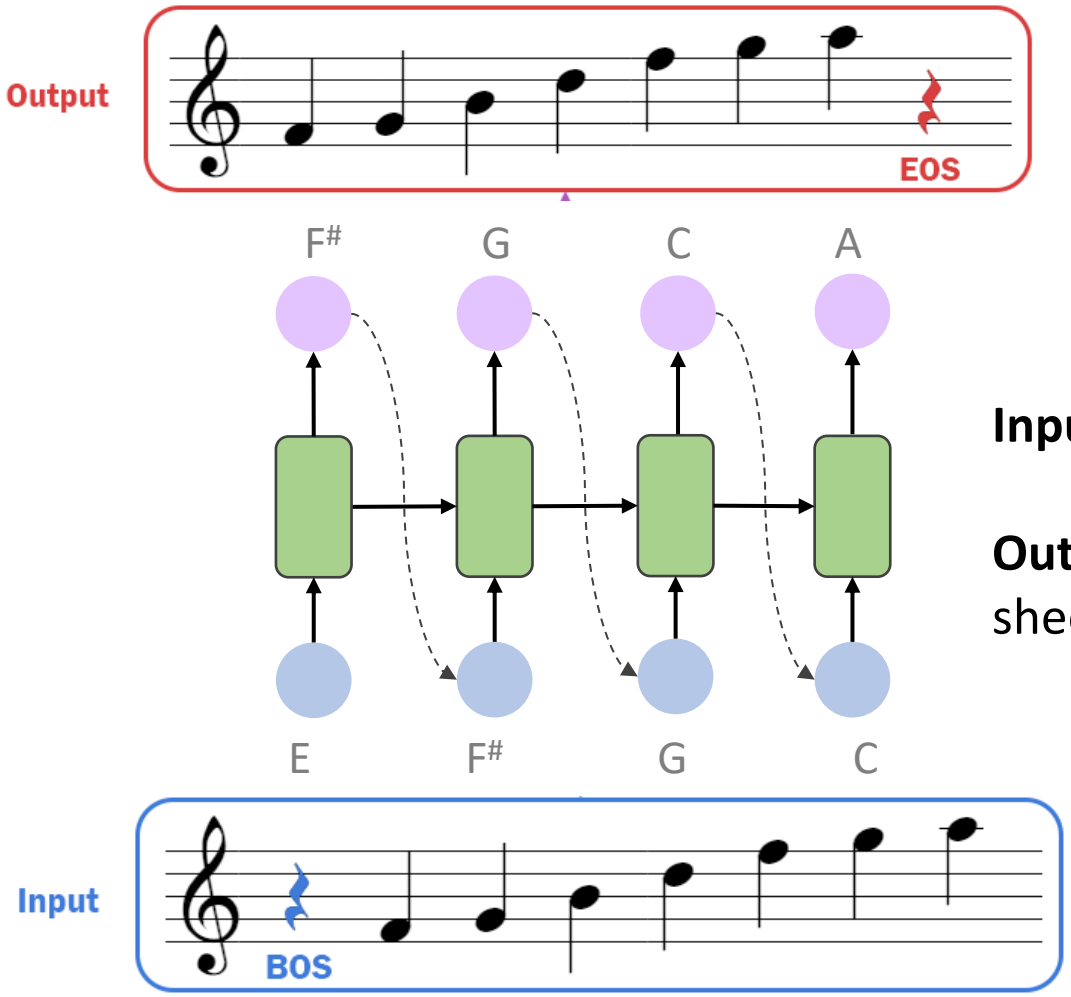
https://gitee.com/wannabe-9/LSTM_poem1

https://zhuanlan.zhihu.com/p/138270447

https://www.kaggle.com/code/jiaminggogogo/ancient-chinese-poem-generation-rnn/notebook



Example Task: Music Generation



Input: sheet music

Output: the next character in sheet music

Example Task: Machine Translation

- Generate a sequence conditioned on the other sequence.

